

作業系統期末考 —— 歷屆考古題解答 (2011–2016)

對應題目見 [期末考古題彙整.md](#)。範圍：Ch5 (排程演算法) → Ch9 (虛擬記憶體)。

壹、CPU 排程

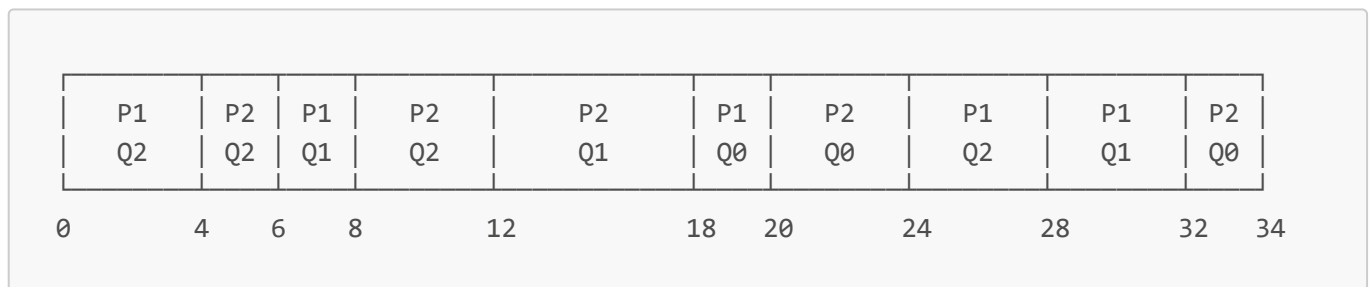
【題型 1】多層反饋佇列甘特圖 ★★

規則複習：

- Q2 (最高)：RR, quantum = 4ms · 新行程進入此層
- Q1 (中)：RR, quantum = 6ms
- Q0 (最低)：FCFS
- 高層行程搶占低層行程；被搶占或用完 quantum 的行程降一層
- I/O 完成後回到 **Q2** (當作新到達)

2012 / 2013 版本：P1 (CPU8, I/O4, CPU8) arr 0；P2 (CPU2, I/O2, CPU16) arr 2

甘特圖：



I/O：P2 t=6→t=8 (2ms)，P1 t=20→t=24 (4ms)

執行步驟說明：

區段	說明
0–4	P1 Q2 · 用完 quantum 4ms, CPU1 剩 4 → 降 Q1
4–6	P2 Q2, CPU1=2ms 跑完 → I/O (t=8 完成)
6–8	P1 Q1 · 跑 2ms, CPU1 剩 2
8–12	P2 I/O 完成回 Q2 搶占 P1 (P1 降 Q0 剩 2)；P2 Q2 跑 4ms · 剩 12 → 降 Q1
12–18	P2 Q1 · 跑 6ms, CPU2 剩 6 → 降 Q0
18–20	P1 Q0, CPU1 剩 2 跑完 → I/O (t=24 完成)
20–24	P2 Q0 · 跑 4ms, CPU2 剩 2

區段	說明
24-28	P1 I/O 完成回 Q2 搶占 P2；P1 Q2 跑 4ms, CPU2 剩 4 → 降 Q1
28-32	P1 Q1 · 跑 4ms, P1 完成 (t=32)
32-34	P2 Q0 · 跑 2ms, P2 完成 (t=34)

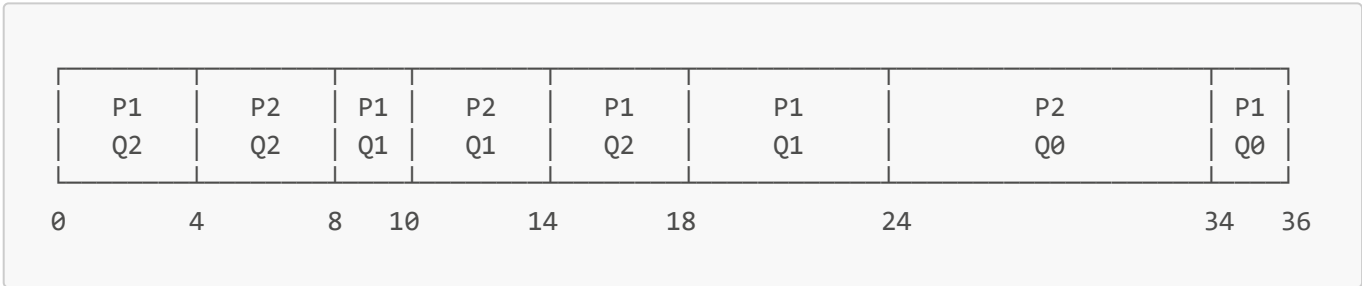
(b) 平均等待時間：

行程	到達	完成	完成時間(Turnaround)	CPU 總長	I/O	等待時間 = TA - CPU - I/O
P1	0	32	32	16	4	12
P2	2	34	32	18	2	12

平均等待時間 = (12 + 12) / 2 = 12 ms

2011 版本：P1 (CPU6, I/O4, CPU12) arr 0；P2 (CPU18) arr 2

甘特圖：



I/O：P1 t=10→t=14 (4ms)，P2 無 I/O

執行步驟說明：

區段	說明					
0-4	P1 Q2 · 跑 4ms, CPU1 剩 2 → 降 Q1					
4-8	P2 Q2 · 跑 4ms, CPU1 剩 14 → 降 Q1					
8-10	P1 Q1 · 跑 2ms, CPU1 完成 → I/O (t=14 完成)					
10-14	P2 Q1 · 跑 4ms ; t=14 P1 I/O 完成搶占 · P2 降 Q0 (剩 10)					
14-18	P1 I/O 完成回 Q2 · 跑 4ms, CPU2 剩 8 → 降 Q1					
18-24	P1 Q1 · 跑 6ms, CPU2 剩 2 → 降 Q0					
24-34	P2 Q0 FCFS · 跑 10ms, P2 完成 (t=34)					
34-36	P1 Q0 · 跑 2ms, P1 完成 (t=36)					
行程	到達	完成	TA	CPU	I/O	等待
P1	0	36	36	18	4	14

行程	到達	完成	TA	CPU	I/O	等待
P2	2	34	32	18	0	14

平均等待時間 = $(14 + 14) / 2 = 14 \text{ ms}$

【題型 2】多處理器負載平衡 (2015)

兩種方法：

1. **Push migration (推移)**：一個特定 kernel 行程週期性檢查各處理器負載，把行程從過載的處理器推到閒置或較不忙的處理器。
2. **Pull migration (拉移)**：閒置的處理器主動從忙碌的處理器拉一個等待中的行程過來執行。

(兩者不互斥；Linux 每 200ms 執行 push，或 run queue 為空時 pull。)

【題型 3】Little's Formula (2015)

Little's formula： $n = \lambda \times W$

- n = 佇列中平均行程數
- λ = 平均到達率 (如 7 行程/秒)
- W = 行程在佇列中的平均等待時間

知道其中兩個可算出第三個。

為何可疑：排隊分析的數學對於複雜演算法和分布難以處理，因此分布常以不切實際的方式定義，可能不準確，計算結果的準確性因而存疑。

【題型 4】等待時間 vs 完成時間 (2016)

- **Waiting time (等待時間)**：行程在 **ready queue** 中等待的總時間。排程演算法只影響這個時間 (不影響執行與 I/O 時間)。
- **Turnaround time (完成時間)**：從行程提交到完全執行完畢所花的總時間 = 在 ready queue 等待 + 在 CPU 執行 + 做 I/O 的所有時間總和。

關係：Turnaround = Waiting + CPU burst + I/O time。

【題型 5】多層佇列選擇題 (2016)

答案：(B) Each queue has its own scheduling algorithm.

- (A) 錯：多層佇列 (multilevel queue) 行程不能在佇列間移動 (能移動的是多層反饋佇列 feedback queue)。
- (C) 錯：最通用的是 multilevel feedback queue。
- (B) 對：每個佇列有自己的排程演算法 (如前景 RR、背景 FCFS)。

【題型 6】飢餓與防止 (2016)

Starvation (飢餓)：在優先權排程中，若系統持續有高優先權行程進入，低優先權行程可能永遠無法執行（無限期被阻塞）。

防止方法 — Aging (老化)：隨著行程在系統中等待時間增加，逐漸提高其優先權（例如每 15 分鐘 +1）。等夠久後，低優先權行程最終會升到最高優先權而被執行。

貳、行程同步

【題型 7】TestAndSet n-process 解 (2011) ★★

(a) 標準答案：

```
do {
    waiting[i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = FALSE;

    // CRITICAL SECTION

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;

    // REMAINDER SECTION
} while (TRUE);
```

(b) 滿足三需求：

- **Mutual Exclusion (互斥)**： P_i 進入臨界區只在 `waiting[i]==false` 或 `key==false`。第一個執行 `TestAndSet` 的行程得到 `key==false`，其餘必須等待。當有行程離開臨界區，只有一個 `waiting[j]` 被設為 `false`。
- **Progress (進展)**：離開臨界區的行程不是把 `lock` 設 `false` 就是把某個 `waiting[j]` 設 `false`，讓等待中的行程能進入。
- **Bounded Waiting (有限等待)**：離開臨界區時以循環順序 ($i+1, i+2, \dots$) 掃描 `waiting` 陣列，任何等待行程最多等 $n-1$ 次即可進入。

【題型 8】Peterson's Solution (2012) ★★

(a) 標準答案：

```

do {
    flag[i] = TRUE;                // entry section
    turn = j;
    while (flag[j] && turn == j);

    // CRITICAL SECTION

    flag[i] = FALSE;              // exit section

    // REMAINDER SECTION
} while (TRUE);

```

(process P_0 時 $i=0, j=1$)

(b) 三需求：

- **Mutual Exclusion**： P_i 進入臨界區僅當 $flag[j]==false$ 或 $turn==i$ 。若兩者都想進入， $flag[i]==flag[j]==true$ ，但 $turn$ 只能是 i 或 j 其一，故只有一個能進入。
- **Progress**：若 P_j 不想進入 ($flag[j]==false$)， P_i 可直接進入。
- **Bounded Waiting**： P_j 離開後設 $flag[j]=false$ ；即使馬上又要進入，它會設 $turn=i$ ，故 P_i 最多等 P_j 進入一次。

【題型 9】Dekker's Solution 證明 (2015) ★★

Mutual Exclusion：行程 P_i 只有在 $flag[j]==false$ (P_j 不在臨界區也不想進) 才會離開 while 進入臨界區；若兩者都想進入，靠 $turn$ 變數決定其一禮讓 (設 $flag[i]=false$ 並忙等 $turn$)，保證不會兩者同時進入。

Progress：若只有 P_i 想進入 ($flag[j]==false$)，它立即進入，不被無關行程阻擋；不在 remainder 的行程才參與決定。

Bounded Waiting：當 P_j 禮讓 ($turn=j$ 時 P_i 把 $flag[i]$ 設 false 並等 $turn$ 變回 i)，離開臨界區的行程會把 $turn$ 設給對方，使對方在有限次內進入，不會無限等待。

【題型 10】第一類 Readers-Writers (2012 / 2016) ★★

Reader's protocol：

```

do {
    wait(mutex);
    readcount++;
    if (readcount == 1)
        wait(wrt);          // 第一個 reader 鎖住 writer
    signal(mutex);

    // reading is performed

```

```
wait(mutex);
readcount--;
if (readcount == 0)
    signal(wrt);    // 最後一個 reader 釋放 writer
signal(mutex);
} while (TRUE);
```

Writer's protocol (2016 加考) :

```
do {
    wait(wrt);

    // writing is performed

    signal(wrt);
} while (TRUE);
```

關鍵：`mutex` 保護 `readcount`；`wrt` 是 writer 的互斥鎖，第一個 reader 進入時 `wait(wrt)`、最後一個 reader 離開時 `signal(wrt)`。

【題型 11】Producer-Consumer with Pthreads (2013) ★★

main 初始化：

```
sem_init(&empty, 0, 5);    // 5 個空 buffer
sem_init(&full, 0, 0);     // 0 個滿 buffer
pthread_mutex_init(&mutex, NULL);
```

producer 的 // insert 部分：

```
sem_wait(&empty);          // 等待有空位
pthread_mutex_lock(&mutex); // 鎖住 buffer

buffer[in] = random_item;
in = (in + 1) % 5;

pthread_mutex_unlock(&mutex); // 解鎖
sem_post(&full);             // 增加滿 buffer 數
```

順序口訣：`wait(empty)` → `lock(mutex)` → 寫入 → `unlock(mutex)` → `signal(full)`。切記 `wait(empty)` 必須在 `lock(mutex)` 之前，否則會死結。

【題型 12】Semaphore 原子性 (2013)

若 `S--` 不是原子的，會被拆成 `reg1=S; reg1=reg1-1; S=reg1`。初始 `S=1`，兩行程 `P0`、`P1` 同時做 `wait()`：

```
T0: P0 執行 reg1 = S           {reg1 = 1}
T1: P0 通過 while (S<=0) 檢查 (S=1>0)
T2: P1 執行 reg2 = S           {reg2 = 1}    ← P1 也讀到 1
T3: P1 通過 while 檢查 (S=1>0)
T4: P0 執行 reg1 = reg1 - 1     {reg1 = 0}
T5: P0 執行 S = reg1           {S = 0}
T6: P1 執行 reg2 = reg2 - 1     {reg2 = 0}
T7: P1 執行 S = reg2           {S = 0}
```

結果：**P0 和 P1 都通過 `wait()` 進入臨界區**，`S` 最終為 0 (應為 -1 或正確阻塞其一)。互斥被破壞 → 證明 `S--` (含測試) 必須是原子操作。

【題型 13】臨界區三需求 / Exit Section 問題 (2014)

(c) 若在 **exit section** 只是「釋放鎖」會有什麼問題？若把 `wait(mutex)` 和 `signal(mutex)` 順序寫反 (在進入時 `signal`、離開時 `wait`)，會使多個行程同時進入臨界區，破壞互斥；若把 `signal` 誤寫成 `wait`，則單一行程也會死結。重點：exit section 必須正確地 `signal(mutex)` 釋放鎖，讓等待行程能進入。

【題型 14】Monitor 條件變數訊號模型 (2015)

兩種 `signal` 後的處理規則：

1. **Signal and wait (訊號並等待)**：發出 `signal` 的行程讓被喚醒的行程立刻執行，自己暫停等待 (直到對方離開 monitor 或再 `wait`)。
2. **Signal and continue (訊號並繼續)**：發出 `signal` 的行程繼續執行，被喚醒的行程等到對方離開 monitor 後才執行。

差異：誰先繼續在 monitor 中執行——前者讓「被喚醒者」先跑，後者讓「發訊號者」先跑。(因 monitor 內同時只能有一個行程活躍。)

【題型 15】Counting Semaphore 選擇題 (2016)

答案：(A) is essentially an integer variable.

- (B) 錯：透過 `wait()` 和 `signal()` 兩個標準操作存取。
- (C) 錯：操作必須原子執行，不可被多執行緒同時修改。
- (D) 錯：可用於控制臨界區存取。

參、死結

【題型 16】銀行家演算法 — (0,4,2,0) from p_1 (2011/2012/2014/2015)

★★

快照：

	Allocation				Max				Need = Max - Alloc			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	0	0	0	0
P1	1	0	0	0	1	7	5	0	0	7	5	0
P2	1	3	5	4	2	3	5	6	1	0	0	2
P3	0	6	3	2	0	6	5	2	0	0	2	0
P4	0	0	1	4	0	6	5	6	0	6	4	2
Available = (1, 5, 2, 0)												

Step 1 : Request₁ (0,4,2,0) ≤ Need₁ (0,7,5,0) ? ☒ 是 **Step 2 :** Request₁ (0,4,2,0) ≤ Available (1,5,2,0) ? ☒ 是

Step 3 : 假設配置：

- Available = (1,5,2,0) - (0,4,2,0) = **(1,1,0,0)**
- Allocation₁ = (1,4,2,0), Need₁ = (0,3,3,0)

安全演算法 (**Work** 從 **(1,1,0,0)** 開始)：

順序	行程	Need ≤ Work?	Work 更新
1	P0	(0,0,0,0) ≤ (1,1,0,0) ☒	+Alloc0 → (1,1,1,2)
2	P2	(1,0,0,2) ≤ (1,1,1,2) ☒	+Alloc2 → (2,4,6,6)
3	P3	(0,0,2,0) ≤ (2,4,6,6) ☒	+Alloc3 → (2,10,9,8)
4	P1	(0,3,3,0) ≤ (2,10,9,8) ☒	+Alloc1 → (3,14,11,8)
5	P4	(0,6,4,2) ≤ (3,14,11,8) ☒	+Alloc4 → (3,14,12,12)

全部 Finish = true → 系統處於安全狀態。結論：請求可以批准 ☒，安全序列 <P0, P2, P3, P1, P4>。

💡 安全序列不唯一：P0、P2 完成後 Work=(2,4,6,6)，此時 P1 與 P3 的 Need 都已滿足，故 <P0, P2, P1, P3, P4> 也是合法安全序列。只要能找到「任一條」安全序列即可判定 SAFE。

【題型 17】銀行家演算法 — 證明 (3,2,1) from p_0 導致 Unsafe (2013)

★★

快照：

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	8	5	3	8	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1

P4 0 0 2 4 3 3 4 3 1
Available = (3, 3, 2)

Step 1 : Request₀ (3,2,1) ≤ Need₀ (8,4,3) ? ☒ **Step 2** : Request₀ (3,2,1) ≤ Available (3,3,2) ? ☒ **Step 3** : 假設配置 :

- Available = (3,3,2) - (3,2,1) = **(0,1,1)**
- Allocation₀ = (3,3,1), Need₀ = (5,2,2)

安全演算法 (**Work = (0,1,1)**) :

嘗試	結果
P0 Need (5,2,2) ≤ (0,1,1)?	✗
P1 Need (1,2,2) ≤ (0,1,1)?	✗
P2 Need (6,0,0) ≤ (0,1,1)?	✗
P3 Need (0,1,1) ≤ (0,1,1)?	☒ Work → (0,1,1)+(2,1,1)=(2,2,2)
P1 Need (1,2,2) ≤ (2,2,2)?	☒ Work → +(2,0,0)=(4,2,2)
P0 (5,2,2)≤(4,2,2)? ✗ P2 (6,0,0)≤(4,2,2)? ✗ P4 (4,3,1)≤(4,2,2)? ✗ 卡住	

P0、P2、P4 無法完成 → 找不到安全序列 → 系統 **UNSAFE**。結論：批准此請求會使系統進入不安全狀態，因此不應批准。

(b) 為何「不一定」死結：不安全狀態不等於死結，只是可能導致死結。若這些行程實際上不會把資源用到 **Max** (提早釋放資源)，系統仍可能順利完成。只有當所有行程都堅持要用到最大需求且互相等待對方持有的資源時，才會真正發生死結。

【題型 18】Unsafe 能否回到 Safe (2011/2012/2014)

可以 (**YES**)。不安全狀態不代表死結。系統可能因為某些行程提早釋放資源、或未真的請求到最大需求，而自然回到安全狀態。不安全只是「系統無法保證一定能避免死結」，並非「一定會死結」。

【題型 19】Unsafe 不一定造成死結 (2015/2016)

- **Safe state** ⊂ 非死結狀態 (一定能找到安全序列讓所有行程完成)。
- **Deadlocked state** ⊂ unsafe state (死結一定不安全)。
- 但並非所有 **unsafe state** 都是死結！

不安全狀態表示系統可能走向死結，但若行程不堅持索取到最大需求、或提早釋放資源，仍可能順利完成而不發生死結。所以 **unsafemay but not necessarily lead to deadlock**。

【題型 20】死結的四個必要條件 (2012/2014/2015)

必須四個同時成立才會死結：

1. **Mutual Exclusion (互斥)**：資源一次只能被一個行程使用。
2. **Hold and Wait (持有並等待)**：行程持有至少一個資源，同時等待取得其他行程持有的資源。
3. **No Preemption (不可搶占)**：資源只能由持有的行程自願釋放，不能強制奪取。
4. **Circular Wait (循環等待)**：存在一組行程 $\{P_0, \dots, P_n\}$ ， P_0 等 P_1 持有的資源、 P_1 等 $P_2, \dots, P_n$ 等 P_0 持有的資源，形成環。

【題型 21】證明資源排序防止 Circular Wait (2016)

方法：對所有資源型別定義一對一函數 $F: R \rightarrow N$ ，要求每個行程只能以 **F 遞增順序** 請求資源（請求 R_j 須滿足 $F(R_j) > F(R_i)$ ， R_i 為目前持有）。

反證法：假設仍發生循環等待，存在環 $\{P_0, P_1, \dots, P_n\}$ ， P_i 等待 P_{i+1} 持有的資源 R_i ， P_n 等待 P_0 持有的 R_n 。

- 因 P_i 持有 R_i 卻請求 R_{i+1} ，依規則 $F(R_i) < F(R_{i+1})$ ，對所有 i 成立。
- 串起來： $F(R_0) < F(R_1) < \dots < F(R_n) < F(R_0)$
- 得到 **$F(R_0) < F(R_0)$** ，矛盾！
- 故循環等待**不可能成立** → 不會死結。

【題型 22】m 資源 n 行程無死結證明 (2014)

給定：m 個同型資源，n 個行程；每個行程最大需求介於 $1 \sim m$ ；所有最大需求總和 $< m + n$ 。

反證法：假設發生死結。死結時所有資源都被分配完（m 個全用光），且每個行程都還需要至少 1 個才能完成。

- 設行程 i 已分配 $Alloc_i$ ，還需要 $Need_i \geq 1$ 。
- 死結時： $\sum Alloc_i = m$ （資源全分配）。
- 又 $\sum Max_i < m + n$ ，即 $\sum (Alloc_i + Need_i) < m + n$ 。
- 代入 $\sum Alloc_i = m$ ： $m + \sum Need_i < m + n \rightarrow \sum Need_i < n$ 。
- 但若死結，n 個行程每個 $Need_i \geq 1$ ，則 $\sum Need_i \geq n$ ，矛盾！
- 故至少有一個行程 **$Need_i = 0$** （已可完成），它完成後釋放資源給其他行程 → 不會死結。

【題型 23】Safe/Unsafe 選擇題 (2016)

答案：(D) An unsafe state may lead to a deadlocked state.

- (A) 錯：safe state 不是死結。
- (B) 錯：safe state 保證**不會**死結。
- (C) 錯：unsafe **不一定是**死結（只是可能）。
- (D) 對。

肆、記憶體管理

【題型 24】頁表大小計算 ★★

公式：頁表大小 = 頁數 × 每筆大小；頁數 = $2^{(\text{邏輯位址位元} - \text{頁內偏移位元})}$ ；頁內偏移位元 = $\log_2(\text{頁大小})$ 。頁大小 4 bytes → 偏移 = 2 bits。

年份	邏輯位址	頁內偏移	頁號位元	頁數	每筆	頁表大小
2011/2013/2014/2015	4-bit	2	$4 - 2 = 2$	$2^2 = 4$	2 B	8 bytes
2012	4-bit	2	2	4	2 B	8 bytes
2016	5-bit	2	$5 - 2 = 3$	$2^3 = 8$	2 B	16 bytes

註：實體記憶體大小 (32/64 bytes) 只影響 frame number 所需位元，不影響頁表筆數；頁表筆數只看「邏輯位址」與「頁大小」。

【題型 25】多層分頁 — 填寫邏輯內容 ★★

解題步驟 (two-level，頁大小 4 bytes、邏輯 4-bit 為例)：

- 邏輯位址 4 bits = p1(1 bit) | p2(1 bit) | offset(2 bits) (two-level 把 2-bit 頁號再切成 p1, p2)。
- outer page table：p1=0 → frame 4；p1=1 → frame 2。這代表內層頁表分別放在實體 frame 4、frame 2。
- 從實體記憶體 frame 4、frame 2 讀出內層頁表內容 (每筆 = 該邏輯頁對應的實體 frame 號)。
- 用「內層頁表的 frame 號 × 頁大小 + offset」找到實體位址，把該實體位址的內容填回對應的邏輯位址。

關鍵觀念：

- frame 號 → 實體位址 = frame 號 × 頁大小 (4)。
- 逐一把每個邏輯頁 (p1,p2 組合) 映射到實體 frame，再把 frame 內 4 個 byte 的內容對應到邏輯位址 0~15。
- 三層分頁 (2012/2016, 5-bit) 同理，頁號切成三段。

2013/2015「配置 frames」變體：

- free-frame list = 5, 0, 2, 7, 3, 4 (Frame 5 第一個)。
- 規則：先配置頁表，再配置行程的頁。
- 依序從 free list 取 frame：第一個 frame 5 給 (內層/頁表結構)，接著依邏輯頁順序取 0, 2, 7, 3, 4 給行程的各頁，並在頁表填入對應 valid bit 與 frame 號。

【題型 26】Address Binding 三時機 (2015)

- Compile time (編譯時期)**：若編譯時就知道行程在記憶體的位置，可產生絕對位址 (absolute code)。
 - 弱點：若起始位置改變，必須重新編譯。
- Load time (載入時期)**：編譯時不知位置，產生可重定位碼 (relocatable code)，載入時才綁定。
 - 弱點：行程一旦載入後不能在記憶體中移動 (移動需重新載入)。
- Execution time (執行時期)**：行程執行中可在記憶體間移動，綁定延遲到執行時，需 MMU 硬體支援。大多數通用 OS 採用此法。

前兩者弱點共通點：邏輯位址 = 實體位址，缺乏執行期移動的彈性。

【題型 27】Compaction 選擇題 (2016)

答案：(C) It is possible only if relocation is dynamic and done at execution time.

- (A) 錯：compaction 須在執行時期動態重定位才可行。
- (B) 錯：compaction 解決的是外部碎裂 (**external fragmentation**) ，不是內部碎裂。
- (C) 對。

伍、虛擬記憶體

【題型 28】頁面置換 — 計算 page fault 數 ★★

所有三年 (**2013/2014/2016**) 的參考字串實際相同： 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6 (4 個 frame)

2013/2016 是給 12-bit 二進位邏輯位址、頁大小 512 bytes (2KB÷4 frame) ，取高 3 bits 為頁號，換算後即為上列字串。

Optimal (最佳，置換「未來最久不用」的頁)

步	參考	命中/錯誤	frames (置換動作)
1	1	F	{1}
2	2	F	{1,2}
3	3	F	{1,2,3}
4	4	F	{1,2,3,4}
5	2	H	
6	1	H	
7	5	F	替換 4 (未來不再用) → {1,2,3,5}
8	6	F	替換 5 (未來不再用) → {1,2,3,6}
9-12	2,1,2,3	H	
13	7	F	替換 1 (最久才用，ref17) → {2,3,6,7}
14-16	6,3,2	H	
17	1	F	替換 7 (未來不用) → {1,2,3,6}
18-20	2,3,6	H	

Optimal page faults = 8

LRU (最近最少使用)

步	參考	命中/錯誤	置換
1-4	1,2,3,4	F×4	{1,2,3,4}
5,6	2,1	H	
7	5	F	替換 3 (LRU)
8	6	F	替換 4 (LRU)
9,10,11	2,1,2	H	
12	3	F	替換 5 (LRU)
13	7	F	替換 6 (LRU)
14	6	F	替換 1 (LRU)
15,16	3,2	H	
17	1	F	替換 7 (LRU)
18,19,20	2,3,6	H	

LRU page faults = 10

結論：Optimal = 8 個 page fault；LRU = 10 個 page fault。

【題型 29】Page Fault 處理流程 (2013/2015/2016)

存取無效頁時 OS 的動作：

1. 檢查該行程的內部表，判斷此參考是真正非法 (invalid) 還是只是還沒載入 (page fault)。
2. 若非法存取 → 終止行程。
3. 若合法但不在記憶體 (page fault) → 找一個 **free frame**。
4. 若沒有 **free frame** → 用頁面置換演算法選一個 victim frame。
 - 檢查 victim 的 **modify bit (dirty bit)**：若被修改過 → 須先寫回磁碟；若沒被修改 → 不必寫回 (省一次 I/O)。
5. 將所需頁從磁碟讀入該 frame，更新頁表與 frame table (valid bit)。
6. 重新執行被中斷的指令。

2015 步驟 1-6 (假設一定有 free frame)：

1. 行程參考一個頁 → 查頁表。
2. 頁表顯示 invalid → 觸發 **trap** 給 OS。
3. OS 在 **backing store (磁碟)** 找到該頁。
4. 把缺少的頁讀入 **free frame**。
5. 重設頁表，將該頁標為 valid 並填入 frame 號。
6. 重新執行剛才造成 trap 的指令 (load M)。

【題型 30】虛擬記憶體優於分頁 (2015)

兩個優點：

1. **程式大小不再受實體記憶體限制**：行程可以只有部分在記憶體中執行，程式可大於實體記憶體。
2. **提高多工度與 CPU 使用率**：因每個行程佔用較少記憶體，可同時執行更多行程；載入/置換所需 I/O 也較少，程式啟動更快。

End of 期末考古題解答。